FUGA DE LA PRISIÓN

La última fuga

Un videojuego de sigilo para demostrar Programación Orientada a Objetos.

4 NIVELES **25 CLASES** **PYTHON + PYGAME**

Jose Fernando Chavez Lozano · Mickele Carlos Torres Velazco
Piero Cemei Romero Meza · Richard Aaron Soriano Cordova



Proyecto parcial · Ciclo IV · 2026-II

Escapar exige observar, decidir y actuar

OBJETIVO DEL PROYECTO

Aplicar POO en una campaña jugable.

01 Salir de la celda

Mantener E durante 2 segundos.

02 Evitar la vigilancia

El ruido y la visión activan a los guardias.

03 Conseguir la llave y escapar

La salida exige haber recogido la llave.



Cuatro niveles, una campaña

01

Pabellón de celdas



Forzar la celda y aprender a evitar patrullas.

02

Lavandería



Nuevas coberturas y dos cámaras.

03

Patio de vigilancia



La llave abre atajos para regresar a la salida.

04

El Alcaide



Sobrevivir al combate y conseguir la última llave.

4 llaves × 100 + 4 salidas × 500 + jefe × 1000 = 3400 puntos

Fuentes: src/nivel.py: ETAPAS · docs/evidencias/

25 clases con responsabilidades definidas



Nivel coordina personajes, mapa, ruido, cámaras, llaves y salida.

Vista dibuja el resultado; las reglas permanecen en los objetos del juego.

Compartir y crear objetos son relaciones distintas

Asociación

Personaje → Nivel

Guarda una referencia para consultar el mapa.

```
self.nivel = nivel
```

Agregación

Nivel ◇— Puntaje

Recibe el mismo puntaje a lo largo de la campaña.

```
self.puntaje = puntaje
```

Composición

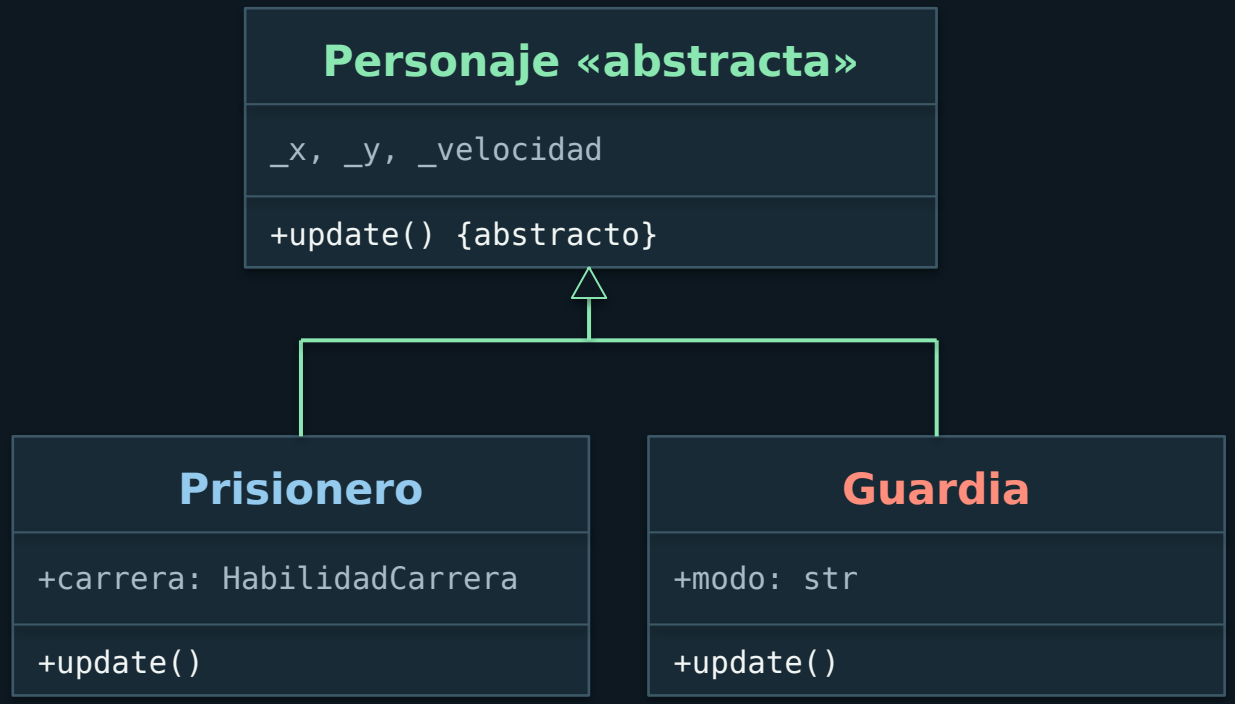
Nivel ◆— Mapa

Crea su propio mapa al cargar el nivel.

```
self.mapa = Mapa(
    PLANOS[indice])
```

La diferencia está en cómo se obtiene el objeto y quién se ocupa de crearlo.

Una misma llamada, dos comportamientos



Teclado, movimiento y ruido de pasos.

Patrulla, investiga y persigue.

HERENCIA + POLIMORFISMO

```
for personaje in self.personajes:
    personaje.update()
```

Nivel usa la misma operación.

Cada clase implementa su comportamiento.

Fuentes: src/personajes.py · src/nivel.py: Nivel.actualizar

El puntaje protege sus propias reglas

ENCAPSULAMIENTO

Los puntos siempre son enteros no negativos.

`_valor`
Dato interno por convención de Python.

`get_valor()`
Consulta el puntaje.

`set_valor(valor)`
Valida el cambio y avisa al panel.

```
def set_valor(self, valor):
    if type(valor) is not int or valor < 0:
        raise ValueError(
            "El puntaje debe ser un entero no negativo.")
    self._valor = valor
    self._notificar()
```

UNA REGLA, UN SOLO LUGAR

Validación + notificación

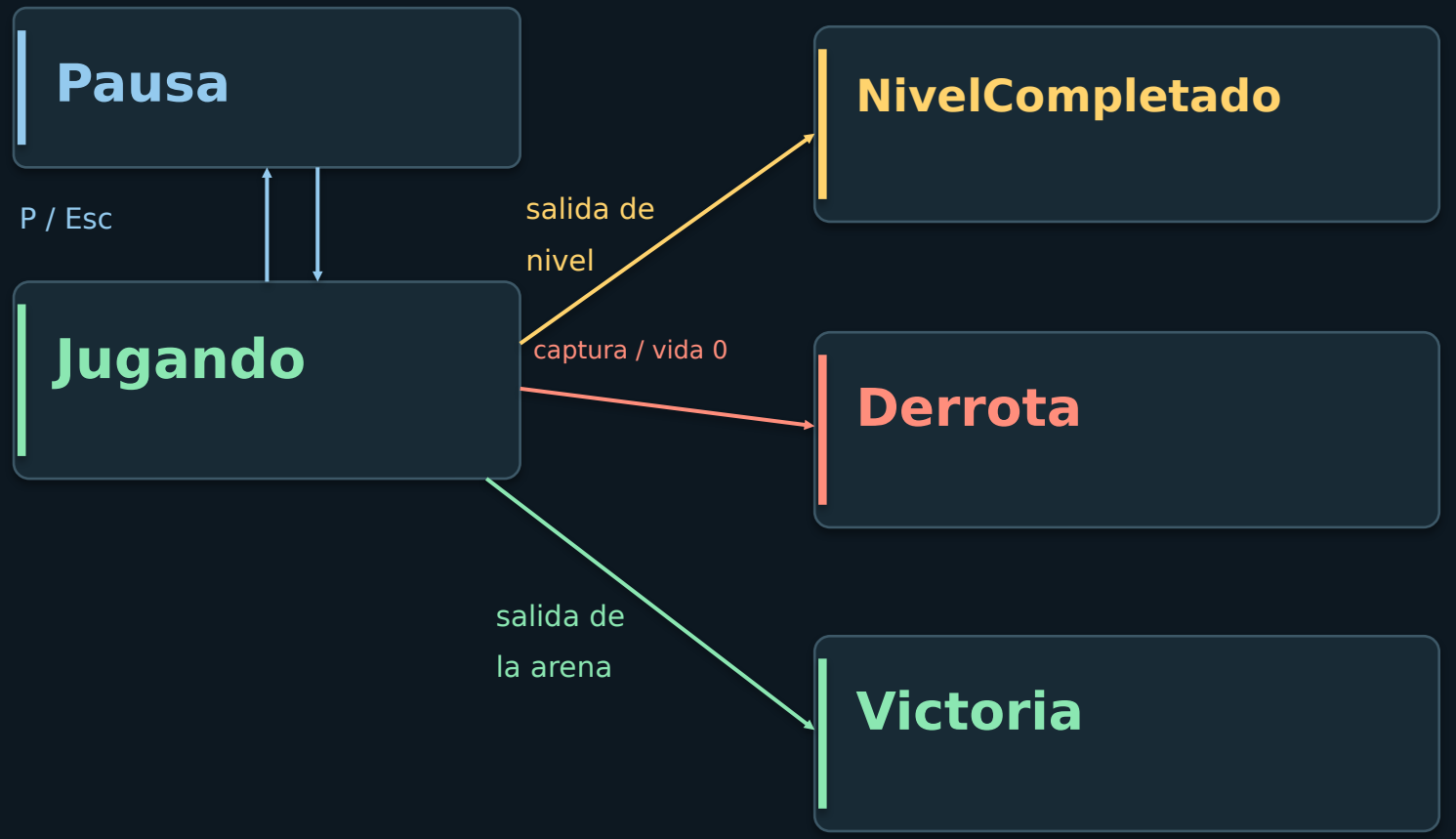
State mantiene coherente cada momento

EstadoJuego

Contrato de entrada y actualización.

Juego delega en el estado activo.

```
self.estado.actualizar(
    dt, teclas)
```



La pausa detiene la simulación: movimiento, vigilancia y combate.

Observer conecta el puntaje con la interfaz

01

Nivel

Ocurre una acción: recoger la llave.

02

Puntaje

Suma 100 puntos y notifica.

03

PanelPuntaje

Recibe el valor actualizado.

```
self.puntaje.suscribir(self.panel)
```

Vista dibuja el valor que recibió el panel.

Puntaje conoce observadores; Vista se ocupa de dibujar.

El jefe cambia de dificultad al perder una capa



3 CAPAS × 72 PUNTOS DE VIDA

FASE	CORONA	ESPERA
01	12	1,45 s
02	40	0,85 s
03	84	0,45 s

La llave aparece al agotar las tres capas.
Cada transición se anuncia durante 1,1 s.

Fuentes: src/combate.py: Alcaide · tests/test_campana.py

La campaña completa está comprobada

54
PRUEBAS APROBADAS

4
NIVELES COMPLETADOS

14
CAPTURAS GENERADAS

¿Qué se comprueba?

Paredes y caminos · ruido y persecución
Puntaje y estados · tres capas del jefe
Coherencia entre clases, guía y UML

MOVIMIENTO Y DISPAROS REALES EN LA SIMULACIÓN



Git conserva la evolución del proyecto

14

COMMITS EN EL HISTORIAL

Planificación, código y documentación quedan registrados.

- `cfaba6e` Plan inicial del proyecto
- `8917f20` Prototipo y pruebas
- `f3cd9fe` Informe, diagramas y guion
- `56f629f` Estado actual del repositorio

En la exposición, cada integrante debe explicar su aporte con evidencia concreta.

Demostración: de la celda a la salida

01

Abrir y moverse

E durante 2 s.

Moverse contra una pared.

02

Generar ruido

Ctrl reduce el ruido.

Shift permite correr.

03

Pausar y continuar

P / Esc pausa.

R reintenta tras perder.

Al recoger la llave: +100. Al salir: +500.

Si la partida no permite llegar al jefe, usar sus capturas para explicar las fases.

La arquitectura sostiene la jugabilidad

Reglas claras.
Objetos con una función.

01 Separar mapa, ruido y nivel mantiene coherentes las reglas.

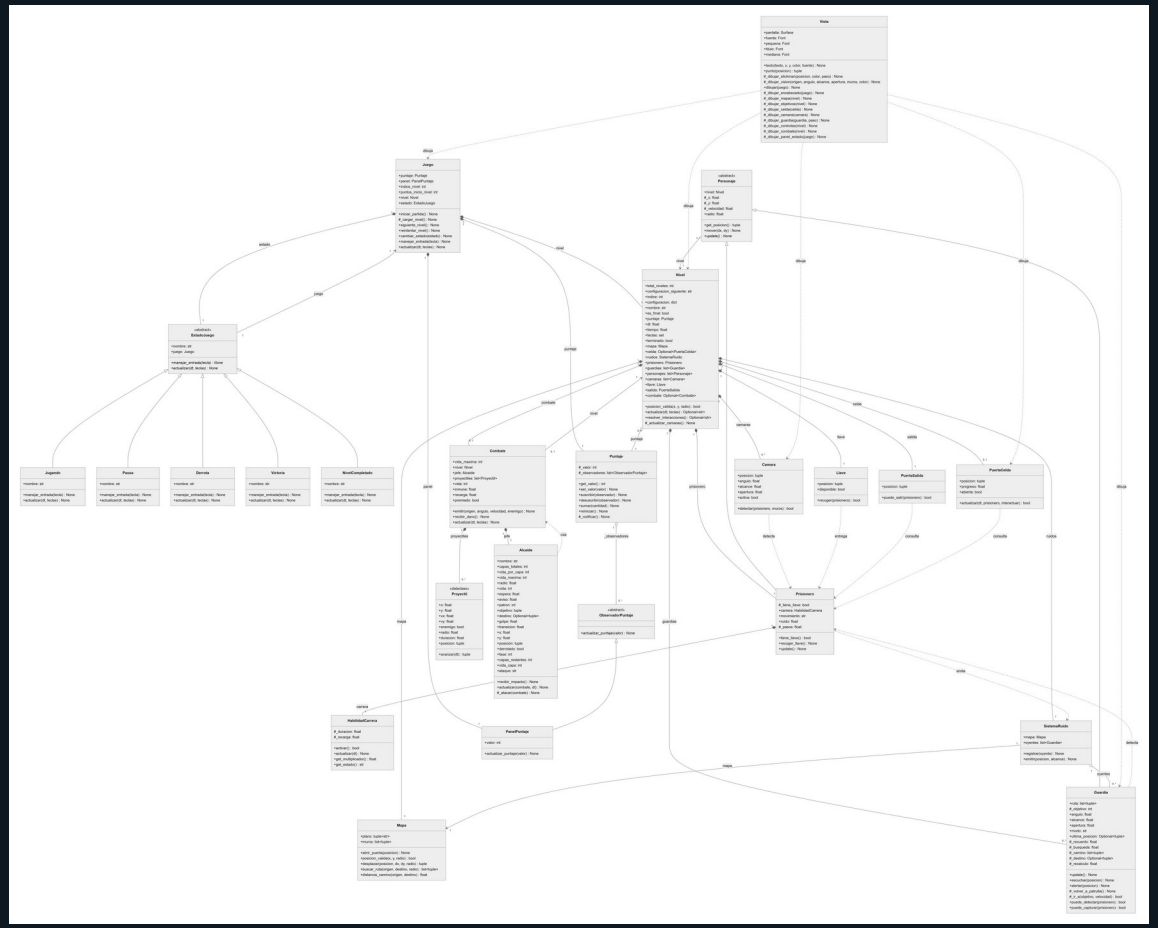
02 Herencia y polimorfismo unifican el tratamiento de personajes.

03 State y Observer coordinan el flujo y la interfaz.

4 niveles
25 clases

Diseño explicado.
Comportamiento comprobado.

Anexo · UML completo de las 25 clases



Cómo leerlo

Triángulo vacío
Apunta a la clase base: herencia.

Rombo negro / blanco
Composición / agregación; junto al contenedor.

Flecha discontinua
Dependencia: utiliza otro objeto.

+ / #
Miembro público / uso interno por convención.

Fuente: docs/diagramas/modelo_parcial.png

Anexo · Respuestas que debemos dominar

¿Por qué Nivel recibe Puntaje?

Para conservar el mismo puntaje al cambiar de nivel.

¿Cómo se entera un guardia?

Por ruido, visión del jugador o alarma de una cámara.

¿Qué detiene la pausa?

La simulación: movimiento, vigilancia y combate.

¿Cuándo aparece la llave de la arena?

Cuando se agotan las tres capas de vida del jefe.

CONTROLES

WASD / flechas Mover
E Abrir la celda
Ctrl Sigilo
Shift Correr
P / Esc Pausar
Espacio / IJKL Disparar
Enter Siguiente nivel